MY LEARNING JOURNEY & MENTORSHIP LOG

Project: AI-Based E-Commerce Product Representation Consistency and Reliability Assessment System
Author: Student Personal Reference & Technical Companion
Mentor: Antigravity (Google DeepMind Agentic Coding Assistant)
Date: September 2026

🌟 Foreword: From Zero Knowledge to Engineering Defense

When we began this project, I told my mentor:

> *"I have 0 knowledge in real-time development... don't even know what an NLI properly is in detail. Please explain each phase and technical terms... work with me like that."*

This document is the unfiltered, personal record of everything we discussed, every technical concept taught, every mistake made, every hidden bug uncovered, and the exact engineering breakthroughs that turned a complex ACL 2025 research paper into a working system on my laptop.

---

1. The Core Philosophy: The Courtroom Analogy

The first breakthrough happened when my mentor explained how to think about this project without using confusing AI jargon:

> **Think of our system as an automated Courtroom:**
> 1. **The Seller Listing is the Defendant's Claim:**
> The seller stands in court and says: *"My product is 100% waterproof polyester, fits up to 24 inches, and does not leak."*
> 2. **The Customer Reviews are the Witnesses:**
> Buyers take the witness stand. One says: *"It kept my hair completely dry."* Another says: *"Water leaked right through the seam."*
> 3. **The S-BERT Retriever is the Court Bailiff:**
> The court has 500 witnesses in the room talking about everything from packaging to shipping to color. The bailiff's job is to find the **only witnesses who actually saw or tested the specific claim** (e.g., finding the people talking about water).
> 4. **The Cross-Encoder NLI Scorer is the Judge:**
> The judge listens to the witness testimony (Premise) and evaluates the seller's claim (Hypothesis). The

> judge issues one of three verdicts:
> * **SUPPORT:** The witness corroborates the claim.
> * **CONTRADICTION:** The witness refutes the claim.
> * **NEUTRAL:** The witness testimony has nothing to do with this claim.
> 5. **The PRI Score is the Final Verdict:**
> If 9 witnesses say it's waterproof and 1 says it leaked, the claim has strong empirical consistency. If 8 witnesses say it leaked, the listing is proven unreliable.

---

2. Why Star Ratings & Sentiment Analysis Failed

Early on, I asked why we couldn't just use standard sentiment analysis or Amazon star ratings. My mentor taught me why:

- **The Star Rating Trap:**
  A customer might give a **5-star rating** because they loved the cute floral pattern, but write in the text: *"Warning: water seeped right through, not waterproof at all."* The star rating records a $+1$ to the product's average score. The fact that the seller lied about waterproofing is completely invisible in the $4.5/5.0$ rating.
- **The Sentiment Trap:**
  Sentiment analysis tools (like VADER or TextBlob) classify text as Positive, Negative, or Neutral. If a customer writes: *"I politely disagree with the size chart; the medium fits like an extra-small, but the color is lovely!"*—sentiment analysis flags it as **Positive/Neutral**. It measures the customer's *mood*, not whether the seller's *dimension claim was accurate*.
- **Our Core Question:**
  We do not ask: *"Is the customer happy?"*
  We ask: **"Did the customer's real experience support or contradict what the seller promised?"**

---

3. The Architecture Debate: Why Not Just Ask Gemini Everything?

During Phase 5, I asked a critical question:

> *"Gemini is an AI, right? It can perform the work of retriever and scorer too. Why don't we just ask Gemini to do everything?"*

My mentor explained that while Gemini *could* do it, doing so would be a terrible engineering decision:
1. **The Cost Bottleneck:**
LLM APIs charge per token. Sending 100 customer reviews for every single bullet point of every single product would cost hundreds of dollars per day.
2. **The Speed & Latency Bottleneck:**

Gemini takes 2 to 3 seconds per API call. If a product has 5 claims and 50 reviews, doing it sequentially takes minutes. Our local S-BERT and NLI models run in **milliseconds** on our own laptop for free.
3. **The Base Paper (PRAISE ACL 2025) Flaw:**
The authors of the base paper explicitly documented that their system made $\approx 2R+1$ LLM API calls per product (over 200 calls for 100 reviews). That was their primary limitation!
4. **Our Hybrid Solution:**
Use the expensive, smart LLM **only once** at the beginning to extract clean claims. Then, use fast, free, local mathematical models (S-BERT and NLI) for the heavy lifting.

If your faculty asks: "*Why did you use a hybrid architecture instead of an end-to-end LLM?*"
You answer: **"We restricted the LLM to claim extraction to minimize API latency and token cost, shifting pairwise retrieval and inference to local, deterministic transformer models."**

---

4. The 4GB RAM Reality & Hardware Constraints

Throughout our project, we worked under real hardware constraints:
* **Operating System:** Windows 10/11
* **Processor:** Intel Core i3
* **RAM:** 4.00 GB (3.80 GB usable)
* **GPU:** No dedicated GPU (integrated graphics only)

When large models run on low RAM, the system swaps memory to the hard drive, freezing Windows. My mentor taught me how real machine learning engineers optimize for edge devices:
1. **Distilled Transformers:**
Instead of using 7-billion parameter models, we chose `all-MiniLM-L6-v2` (80MB) and `nli-MiniLM2-L6-H768` (120MB). Both models combined take less than 300MB of RAM.
2. **Sequential Inference:**
We process claims sequentially, allowing PyTorch tensors to clear memory between loops.
3. **Persistent Model Caching:**
Streamlit reruns scripts on every click. By adding `@st.cache_resource`, the AI models stay loaded in memory once, dropping the run time from 35 seconds down to seconds.

---

5. The Debugging Battles: What Broke and What We Learned

This was the most valuable part of my mentorship. We ran into real bugs that no textbook warned us about, and fixing them taught me real NLP.

Battle 1: The "Context Overload" Problem

- **The Symptom:**
  When we first ran the pipeline on the Shower Cap, the dashboard output showed **0% Support, 0% Contradiction, and 98% Neutral** across almost every claim.
- **The Investigation:**
  We wrote `src/debug_nli.py` and printed out the raw model scores. The reviews were 150–200 words long paragraphs.
- **The Lesson:**
  Small NLI cross-encoders have a limited sequence capacity. When fed a 10-word claim and a 200-word personal essay (talking about someone's shower routine, hair length, and dollar store shopping), the attention heads get overwhelmed. The model cannot locate the specific clause and defaults to Neutral.
- **The Fix:**
  Sentence Chunking. We split every review into single sentences on periods (`.`). Now S-BERT retrieves the *exact sentence* (e.g., "*keeps my hair dry*"), and NLI evaluates that isolated clause with 94% confidence.

Battle 2: The "Robot Lawyer" Dilemma (Opinions vs. Facts)

- **The Symptom:**
  When testing the shower cap, the sentence "*I love this shower cap*" was classified as a **CONTRADICTION** with 90% confidence!
- **The User's Realization:**
  I protested: "*If the reviewer loves the cap, how can that be a contradiction? A human would never think that!*"
- **The Lesson:**
  NLI models are literal mathematical engines trained on strict logic. The seller claimed: "*The cap is completely waterproof.*" The reviewer said: "*I love this shower cap.*"
  The robot lawyer thinks: "*Does loving an item logically guarantee that it is waterproof? No. People love non-waterproof hats too.*" Because there is zero logical entailment, the small model drifted into false contradiction.
- **The Fix:**
  Fact vs. Opinion Filtering. We implemented `_is_factual()` in `pipeline.py`. If a sentence is short and consists purely of subjective phrases ("*i love*", "*great product*", "*worst purchase*"), it is stripped out. Only sentences containing verifiable attribute words ("*dry*", "*water*", "*fits*", "*zipper*", "*broke*") reach the NLI model.

Battle 3: The Backwards NLI Bug (Directional Logic)

- **The Symptom:**
  Even after chunking sentences, our support scores were unusually low.
- **The Investigation:**
  We inspected line 27 of `src/nli_scorer.py`:
  `python result = self.classifier({"text": claim, "text_pair": review_evidence})`

- **The Revelation:**
  In cross-encoders, `text` is the **Premise** (what is accepted as true), and `text_pair` is the **Hypothesis** (what we are testing).
  By passing `text=claim` and `text_pair=review`, we were asking:
  "If the seller promises a waterproof cap, does that prove this customer stayed dry?"
  The AI answered: *"No, a seller promise does not prove an individual outcome!"* $\rightarrow$ **False Neutral!**
- **The Fix:**
  We swapped the order:
  `python result = self.classifier({"text": review_evidence, "text_pair": claim})`
  Now we ask: *"Given that the customer experienced X, does that prove or disprove the seller's claim Y?"*
  Immediately, Support jumped to 90%+!

Battle 4: The 503 API Server Crash & The Cache Strategy

- **The Symptom:**
  While testing, Google's AI Studio returned: `503 UNAVAILABLE: This model is currently experiencing high demand.` The whole pipeline crashed.
- **The Lesson:**
  Free-tier cloud APIs are inherently unreliable for live classroom presentations. If Google's servers are busy in America during my viva, my presentation will crash on stage.
- **The Fix:**
  We built `data/sample/cached_claims.json`. We pre-extracted clean atomic claims for our 5 sample products and saved them locally. The pipeline checks the local cache first. If found, it loads in **0 milliseconds with 0 API calls**. The system is completely offline-ready and crash-proof.

Battle 5: The Dirty DOM Scrape Problem

- **The Symptom:**
  In Manual Input mode, when I copy-pasted a real Amazon jacket page, the system output bizarre contradictions:
- Claim: *"Manufacturer J B Fashion"*
- Contradicting Evidence: *"TAGAS Men's Regular Fit Bomber Jacket"*
- **The Lesson:**
  When a human copies an Amazon page, they inadvertently copy sponsored ads, related product carousels, pricing tables, delivery dates, and reviewer names. The AI was trying to check if a jacket's weight was contradicted by a sponsored ad title!
- **The Fix:**
  We wrote regex noise cleaners (`_clean_seller_input()` and `_clean_review_input()`) that strip sponsored brand text, ASINs, dates, and star ratings, keeping only product sentences.

Battle 6: The Cajun Seasoning Mismatch

- **The Symptom:**
  When testing Product 2 (Cajun Seasoning Base), the PRI score was 0/100, and all reviews were Neutral.
- **The Discovery:**
  We opened `data/sample/sample_products.json` and read the reviews for Product 2. The reviews were talking about **tropical leaves, mousepads, clay ornaments, and graduation charms!**
- **The Takeaway:**
  The Hugging Face dataset download had corrupted review mappings. The AI model wasn't broken—it was actually doing its job properly! It saw that tropical leaves have nothing to do with Cajun seasoning, so it correctly refused to score them as Support.
  Lesson: "*Garbage in, garbage out.*" Data quality is 90% of machine learning success.

---

6. The Visual Consistency Module: Pure Pixel Math

When discussing the visual module, I worried that our 4GB laptop could not handle heavy vision models like OpenAI CLIP. My mentor taught me that we could achieve visual discrepancy checking using **OpenCV pixel mathematics**:

1. **CIE LAB Color Space:**
   Standard RGB color distance fails because it treats red, green, and blue as mathematical cubes, ignoring how human eyes perceive brightness. CIE LAB separates Lightness (L^*) from color opponents (a^* green-red, b^* blue-yellow). By calculating the Euclidean distance (ΔE), our script (`src/visual_scorer.py`) detected a green-vs-red shirt discrepancy instantly with 0% deep learning overhead.
2. **Structural Similarity (SSIM):**
   If a seller advertises a round smartwatch and sends a square smartwatch, SSIM measures edge and contour distortion on black-and-white gradients.
3. **Review I Framing:**
   We implemented this as a standalone proof-of-concept script. In our presentation, we honestly state: "*Visual consistency is tested in an experimental standalone OpenCV prototype; live pipeline integration will take place in Review II.*"

---

7. How to Think Like an Engineer During My Viva

My mentor left me with three rules that changed my confidence completely:

1. **Never apologize for a developmental finding.**
   If the faculty asks: "*Why did your early pipeline give 0/100 on long reviews?*"
   Do not say: "*Our code had a bug.*"
   Say: **"Early testing revealed the context overload problem in transformer cross-encoders when**

handling 200-word review paragraphs. We engineered a sentence-chunking preprocessor to isolate atomic clauses, which restored classification confidence to over 90%."

2. **Never claim fake product detection.**
   Our project is an **Empirical Representation Consistency Audit**. We audit digital claims against digital evidence. We do not do criminal fraud detection.
3. **Know the difference between Similarity and Inference.**
   * S-BERT tells you: "*Are these two sentences talking about the same topic?*"
   * NLI tells you: "*Does sentence A prove or disprove sentence B?*"

---

8. Summary of What We Achieved Together

Milestone	Where We Started	Where We Ended
Conceptual Understanding	Didn't know what NLI meant.	Can explain cross-attention, directional entailment, and bi-encoders.
Codebase	Empty repository with leftover phishing files.	Clean, structured Python virtual environment, modular `src/`, `data/`, `app/`.
Pipeline	Theoretical idea from a research paper.	Working end-to-end prototype running locally on a 4GB i3 laptop.
User Interface	Terminal print statements.	Interactive Streamlit dashboard with dual modes and live evidence expanders.
Academic Defense	Scared of failing Review I.	Armed with 50 viva Q&As, a 6-slide master PPT, and an authoritative source pack.

This document is my personal proof that with clear mentorship, patience, and rigorous debugging, even the most complex AI research can be mastered from the ground up.